Polymarket bot: why it isn't trading, and what to do about it

Date: 2026-10-10. Scope: read-only investigation of `d:\neststock\scripts\polymarket_bot`. Nothing in the repo was changed. No orders were placed.

Short answer

Three separate things stop the bot from trading. Fixing the first two doesn't help unless the third is dealt with.

1. **The process is dead.** Nothing is running. The last heartbeat in `dashboard_state.json` was 2026-10-09 01:30. Nothing auto-starts the bot: no scheduled task, no Startup entry, and the supervisor in `start_bot.py` only restarts its children, not itself. Logs go only to a hidden console, so there's no record of why it died.
2. **It's in paper mode.** `execution_mode` is `"Paper Trading"`. In that mode `LiveExecutor.execute_arbitrage` only simulates the trade (paper_trader.py:5267-5270).
3. **The core strategy can't fire on Polymarket.** Polymarket uses one shared book per market, so the NO ask is the YES bid seen from the other side. `ask_yes + ask_no = 1 + spread`, which is always at least $1.001. Measured on 360 live markets today, the minimum sum was 1.001 and none were under 1.00.

Evidence

Live market measurement (measured, script below)

Check	Result
Two-outcome markets with both books	360
Lowest YES+NO ask sum	1.001
Markets with sum < 1.00	0
Most common sum	1.01 (155 markets), which is just the 1-cent spread
negRisk events fully quoted	22 of 33 checked
negRisk events with sum of YES asks < 1.00	5

The negRisk hits were Republican Nominee 2028 (0.854, 43 legs), Presidential Winner 2028 (0.932, 54 legs), Democratic Nominee 2028 (0.933, 53 legs), Israel PM (0.947, 19 legs), and one soccer match (0.98, 3 legs, 20 shares deep). The long-dated ones are probably "augmented" negRisk events where an unnamed "Other" outcome can win. If so, buying every listed YES isn't risk-free, and the capital is locked until 2028 (inferred from the Polymarket docs). The soccer match roughly breaks even after the 5% sports taker fee.

Script: `C:\Users\redwi\.gemini\antigravity\brain\74d20cd8-160d-489f-8663-614775e7a8e7\scratch\market_reality.py`.

Why the bot's own paths reject everything (from reading the code)

- **The taker path** needs `edge >= max(min_edge, 2*tick + fee)` (paper_trader.py:2447-2449). On 1-cent books that means a YES+NO sum near 0.979. Measured minimum is 1.001.
- **The maker-then-taker path is off below $100 capital** (lines 3029, 3371, 5335). Capital is $69.31. The check at 5335 returns without logging a miss, so the dashboard never shows it.
- **The maker price is always below the best bid.** It's capped at `1 - ask_no*(1+fee) - min_edge`. Since `ask_no = 1 - bid_yes`, that works out to `bid_yes - edge - fee`. The order only fills when the price is falling through it. By then the other leg costs more, so the second leg fails. That fits `primary_bottleneck = MAKER_TIMEOUT`, the 16,869 killed orders, and the stranded YES position (inferred from the code plus the measured book structure).

What the dashboard numbers actually mean

- **461,857 CONCURRENCY_EXHAUSTED** is mostly a paper-mode artifact. Every candidate in a websocket frame is tried in turn. Once the slots fill, the rest of the frame logs a miss, and nothing deduplicates them. The last 50 misses all share one timestamp.
- **In live mode the slots really do leak.** `recycle_collateral` returns early in live mode (paper_trader.py:1707-1709). A hedged pair then holds its slot until the market resolves, possibly for weeks.
- **The "adaptive" fix for concurrency does nothing.** It writes `max_concurrent_positions`, but `_apply_to_simulator` never applies that key, and `auto_apply` is False.
- **CLOB_ORDER_KILLED is a catch-all** for every maker-taker failure. No error message is stored, so the 16,869 count can't be broken down.

Execution bugs that would bite in live mode

Ranked by how likely each is to lose money or block trades. All found by reading code; none reproduced live.

1. **`neg_risk` is always False on binary trades.** `market_token_map` has no top-level `neg_risk` key, so `m_info.get('neg_risk', False)` (line 2538) always returns False. That False is passed explicitly to `PartialCreateOrderOptions` (concurrent_leg_executor.py:213), which overrides the client's own lookup. On neg-risk markets, orders are signed for the wrong exchange contract.
2. **The cheap leg is below the $1 minimum for marketable orders.** Example: 7 shares at $0.046 is $0.32. The cheap leg gets rejected while the expensive leg fills, which leaves an orphan (inferred from the CLOB rules).
3. **`purge_all_orders` calls `cancel_all()`** at startup, at shutdown, and mid-run on cancel errors (order_reaper.py:352). That wipes the protective rollback sell orders.
4. **No merge after a matched pair.** The dual-taker path returns before the merge code. The merge code signs from the EOA instead of the deposit wallet, and it doesn't use the NegRiskAdapter for neg-risk markets. Capital stays locked until resolution.
5. **Leg-out handling.** It sells right after a BUY that's MINED but not confirmed, then retries only 3x1s. A rollback sets only a 30s cooldown, so the bot can re-enter and fail the same way again.
6. **Book data issues.** Depth in shares is treated as USD (lines 2484, 2515). The legacy hedge reads REST `asks[0]` / `bids[0]` as best price, but REST returns the best level last.
7. **Zero safety margin.** `min_edge` is set to the opportunity's own edge (lines 5219, 5291).

Ops findings

- Logging uses `basicConfig` with no file handler (paper_trader.py:56). Children inherit a console that `start_bot_silent.vbs` hides.
- `cleanup_orphaned_bot_processes` runs `taskkill /F /T` on the PID in `paper_trader.pid`. Windows reuses PIDs, so a stale file could kill an unrelated process tree.
- `requirements.txt` pins both `py-clob-client==0.34.6` and `py-clob-client-v2==1.2.0`. The live client is v2.
- Tests: 327 passed, 1 failed. The failure (`test_emergency_dump_never_forces_market_exit_into_wide_bid`) is a stale assertion. The code now passes `tick_size` and `neg_risk` to `_post_sell_order`, and the test doesn't expect them.

Open position

18 YES shares on "Israel x Iran ceasefire continues through November 15?" at avg 0.69. Current value $12.33, which is -$0.09. There's no NO leg. This is directional exposure until 2026-11-16.

How other bots do this

Bot	Strategy	Relevant takeaway
warproxxx/poly-maker	Maker-only two-sided quoting, ranked by reward density	Post-only quotes, merges YES+NO through CTF or NegRiskAdapter via the relayer, websocket plus heartbeat. No audited profit; the README warns it can lose money.
Polymarket/poly-market-maker	Bands / AMM keeper near mid	Labeled experimental. Treats BUY YES as equal to SELL NO at the complement price, which is the same shared-book fact.
ImMike/polymarket-arbitrage	Bundle and cross-venue arb	Its profit figure comes from simulated data. Its README says real opportunities are rare.
arXiv 2508.03474	Study of $39.6M extracted Apr 2024 to Apr 2025	About 70% came from inside negRisk events (buy-NO $17.3M, buy-YES $11.1M). That period had no taker fees.

Current fees (docs) are taker only: `fee = C * rate * p * (1-p)`. Rates are crypto 0.07, sports 0.05, politics 0.04, and geopolitics 0. Makers pay nothing and get 15-25% rebates.

Recommendations

Ranked by expected value. The first two are calls you need to make. The rest are engineering.

1. **Stop treating single-market YES+NO parity as the strategy.** Tuning thresholds won't help. The opportunity doesn't exist on this exchange. Most of `paper_trader.py` (322 KB) is built around a trade that can't fire.
2. **Decide what to do with the 18-share YES position.** You can hold it to resolution, or sell it at about 0.685 for roughly a $0.09 loss.
3. **If you want to keep doing arbitrage, go negRisk-only:**
4. Price fees per leg as `rate * p * (1-p)`. The scanner currently uses `sum(ask) * (1 + fee)`.
5. Add the buy-all-NO basket.

6. Skip augmented events, or include the "Other" outcome in the sum.
7. Prefer fee-free geopolitics events and short-dated events.
8. Require at least 3 cents of edge after fees.
9. Merge through the NegRiskAdapter right away.

At $70 the 5-share minimum per leg caps you at events with only a few outcomes. 4. **If you want steady small income instead, switch to passive two-sided quoting, poly-maker style.** Be realistic about what $70 can do. The reward pool worth having that I measured (Texas vs Oklahoma, $1,714/day) needs 1,000 shares per side, about $790. The 50-share pools pay close to nothing. Expect cents a day, with real adverse-selection risk. 5. **Fix these whichever strategy you pick:** - The `neg_risk` flag. - A $1-notional check on each leg. - Scope `cancel_all()` down to registered order IDs. - Merge after every matched pair. - Free slots on merge or after the hold period. - Store the error message on each missed trade. - Dedupe the miss counter. 6. **Ops:** - Add a rotating file log. - Register a Task Scheduler task that starts at logon and restarts on failure. - Check the process name before the PID-file `taskkill`. - Fix the stale test.

Method

This was a five-worker swarm: gate funnel, live market check, prior-art research, process forensics, and execution audit. Three workers had no shell, so the coordinator ran the shell checks, the test suite, and the 360-market probe directly. Worker claims were spot-checked against the source: the $100 gates, the live-mode recycle bypass, the `2*tick+fee` floor, the `neg_risk` lookup, and `requirements.txt`.